Introdução

Introdução à Estrutura de Dados

0004617A/4710A – Estrutura de Dados I

Prof. Me. Giovani Candido

Prof. Me. Luiz Fernando Sementille

BCC/BSI - Bauru

Definições

Tipo Abstrato de Dados (TAD)

- Um TAD é uma **especificação** constituída por:
  - Um **conjunto de variáveis**, que são utilizadas para armazenar informação.
  - Um **conjunto de operações** bem definidas sobre essas variáveis.
- Trata-se de uma interface que se preocupa com **"o que fazer"**, e não com o "como fazer".
- Especifica como os valores podem ser relacionados e manipulados, sem se preocupar com os detalhes internos de uma eventual implementação.
- Para o usuário de um TAD, o que importa é o comportamento, isto é, o efeito das operações, uma vez que a implementação pode ser feita de várias formas.

Interface x Implementação

```c
initialize(&head);

do {
    printf("Enter new values: (-1 to stop): ");
    scanf("%d", &item);

    if (item == -1) {
        break;
    }

    insert_back(&head, item);

    show(head);
} while(item != -1);
```

```c
int insert_back(Node *head, int item) {
    Node new_node = create_node(item);

    if (new_node == NULL) {
        return 0;
    }

    if (*head == NULL) {
        *head = new_node;
        return 1;
    }

    Node temp = *head;

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = new_node;

    return 1;
}
```

TAD - Números Racionais

Exemplo: Um TAD pode ser um tipo para lidar com números racionais.

- Em termos de **valores**, teríamos:
  - **Duas variáveis:** o numerador e o denominador da fração que representa o número racional armazenado.
  - **Uma condição do TAD:** o segundo inteiro (denominador) deve ser diferente de 0.

- Em termos de **operações**, poderíamos ter funções como:
  - **make_rational** (criação)
  - **add** (adição)
  - **mult** (multiplicação)
  - **equal** (teste de igualdade)

TAD - Conjunto

Exemplo: TAD pode ser um conjunto matemático de inteiros.

- Embora um TAD não precise estar associado a nenhuma linguagem de programação, podemos utilizar a sintaxe da linguagem C para ajudar na definição do TAD.
- Como definição dos valores, poderíamos ter:

```
#define MAX_LEN 100

typedef struct set_type {
    int elements[MAX_LEN];
    int cardinality;
    int iterator;
} Set;
```

TAD - Conjunto

- Podemos também definir as seguintes operações de acesso:

```
// Funções de acesso

// Cria um conjunto vazio, deve ser chamado antes de qualquer operação no conjunto
void initialize_set(Set *s) ;

// Escolhe um elemento qualquer do conjunto para ser removido
void remove_one_element(Set *s, int *element);

// Inicializa o iterador que sera usado na função increment_iterator
void initialize_iterator (Set *s);

// Incrementa o iterador, retorna 0 se acabou conjunto
int increment_iterator(Set *s);
```

TAD - Conjunto

- Além de outras operações de manipulação do conjunto:

```
// Funções de manipulação

int is_empty(s); // Retorna 1 se o conjunto s é vazio e 0 caso contrário
int cardinality(Set s); // Retorna a cardinalidade do conjunto s
void insert(int x, Set *s); // Insere o elemento x no conjunto s, mantém os elementos ordenados
void remove(int *x, Set *s); // Remove o elemento x do conjunto s
Set set_union(Set s1, Set s2); // Obtém a união dos conjuntos s1 e s2
Set set_intersection(Set s1, Set s2); // Obtém a intersecção dos conjuntos s1 e s2
Set set_difference(Set s1, Set s2); // Obtém a diferença dos conjuntos s1 e s2 (s1 - s2)
int belongs(int x, Set s); // Retorna 1 se x pertence ao conjunto s e 0 caso contrário
int equal(Set s1, Set s2); // Retorna 1 se o conjunto s1 = s2 e false caso contrário
int contained(Set s1, Set s2); // Retorna 1 se o conjunto s1 está contido no conjunto s2 e 0 caso contrário
Set copy_set(Set s); // Copia os elementos do conjunto s1 para outro conjunto
```

Estrutura de Dados

- É uma forma de **organizar** e **armazenar** dados na memória, para que possam ser **acessados** e **manipulados** de maneira eficiente durante a execução de um programa.
- Constitui a base para a execução de algoritmos, e sua escolha depende do contexto, pois é necessário avaliar as operações necessárias e o custo delas.
- A estrutura de dados preocupa-se com o **"como fazer"**, sendo a implementação de um TAD, responsável pelos detalhes internos não especificados pela interface.
- Um TAD pode ser implementado de diferentes maneiras, cada uma com pontos fortes e limitações, sendo mais adequada a determinados tipos de problemas.

Listas Lineares

Listas

- Estrutura de dados que **agrupa um conjunto de valores organizados linearmente** que, de alguma forma, se relacionam entre si.
  - Devido à ordem linear, são também denominadas **listas lineares (linear lists)**.
- Podem armazenar uma sequência numérica, itens em um estoque etc., sendo capazes de conter inúmeros tipos de dados, incluindo estruturas (*struct*).
- As operações mais frequentes em listas são: **busca**, **inclusão de um elemento** e **remoção de um elemento**, que aliás ocorrem em muitas estruturas.

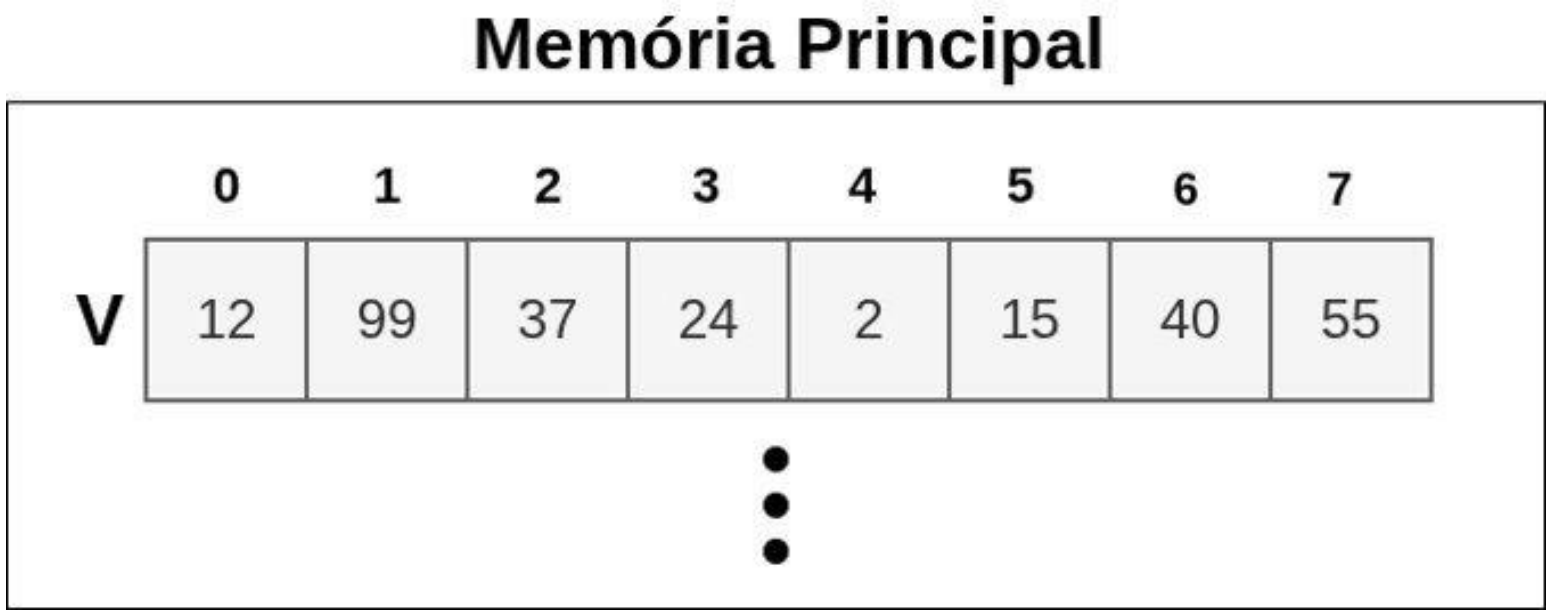
Listas

- Outras operações importantes são:
  - **Alteração de um elemento;**
  - **Ordenação dos elementos;**
  - **Determinação do primeiro ou último elemento;**
  - **Combinação de duas ou mais listas;**
  - **Determinação da cardinalidade.**
- Essa estrutura serve de base para a implementação de outras que veremos durante o curso, com pilha e fila sendo casos especiais de lista.
- São divididas de acordo com a organização em memória em: **sequencial** e **encadeada**.

Alocação Sequencial

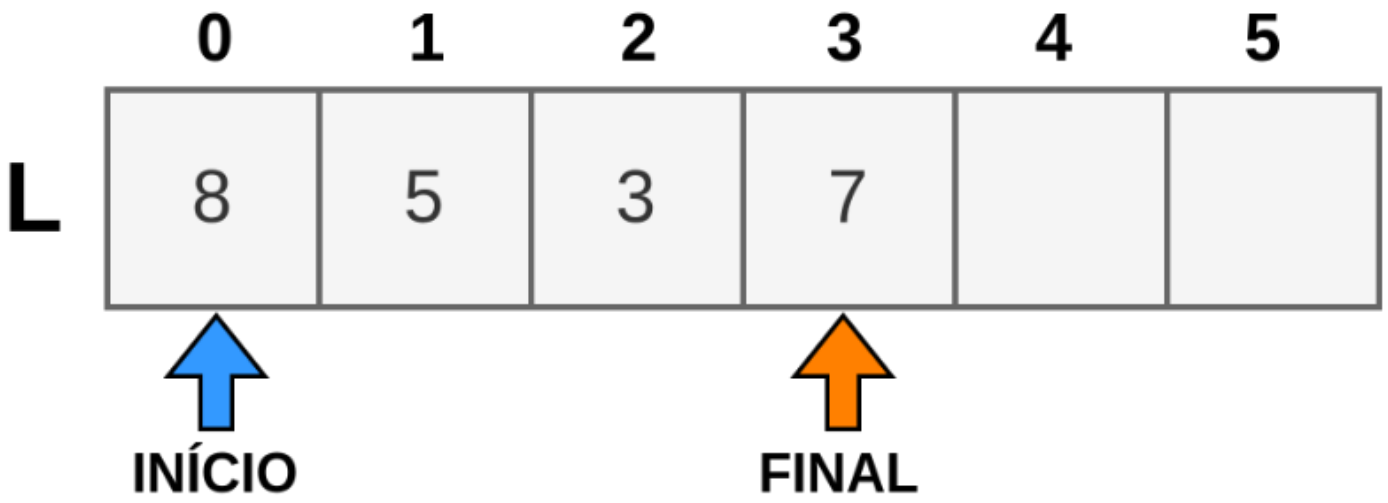
Listas Estáticas

- É a maneira mais simples de manter uma lista na memória do computador, pois seus nós ocupam **posições contíguas**, daí a denominação sequencial.



Listas Estáticas

- Esse tipo específico de lista é implementada, em C, utilizando um **vetor (*array*)**.
- Como a implementação desse tipo de alocação geralmente envolve a reserva prévia de memória, pode ser considerada uma **alocação estática (static lists)**.

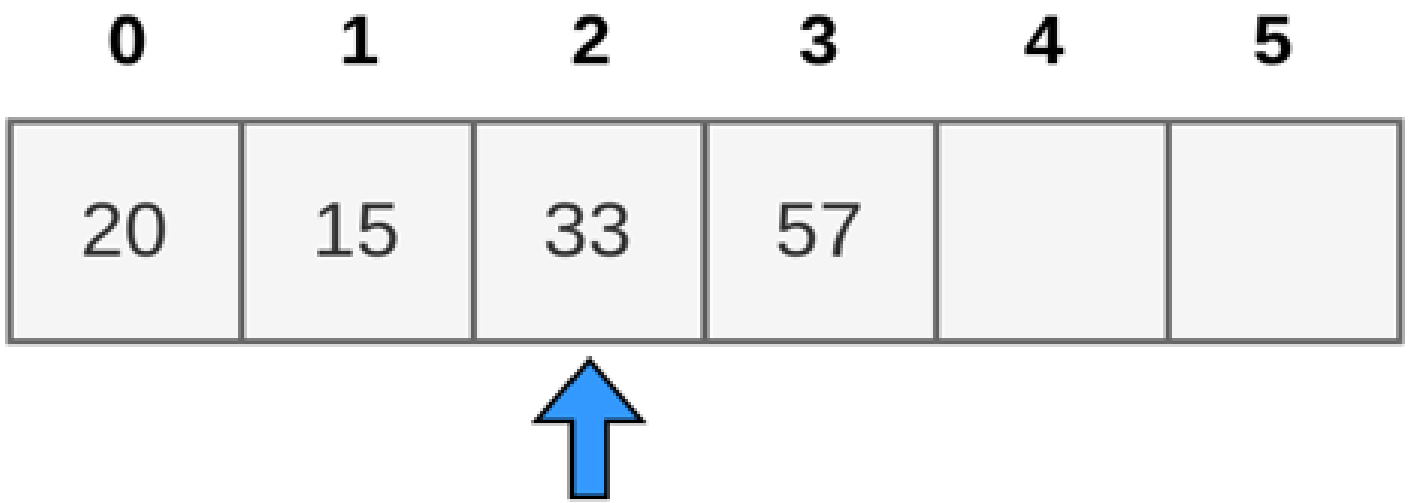


Listas Estáticas

- Devido à forma como os dados estão dispostos na memória, o acesso aos dados é **direto** (ou aleatório), realizado **por meio de índices.**
- A inserção ou remoção de um nó não ocorre de fato, mas é **simulada por meio de variáveis** indicando os limites da memória realmente utilizada.
- Dessa forma, a inserção e remoção de nós ao final da lista é eficiente, porém inserir e remover no início ou em posições arbitrárias acarreta em deslocamentos.
- **Desvantagem:** Se o usuário precisar de mais espaço, é preciso fazer uma nova alocação e copiar os elementos.

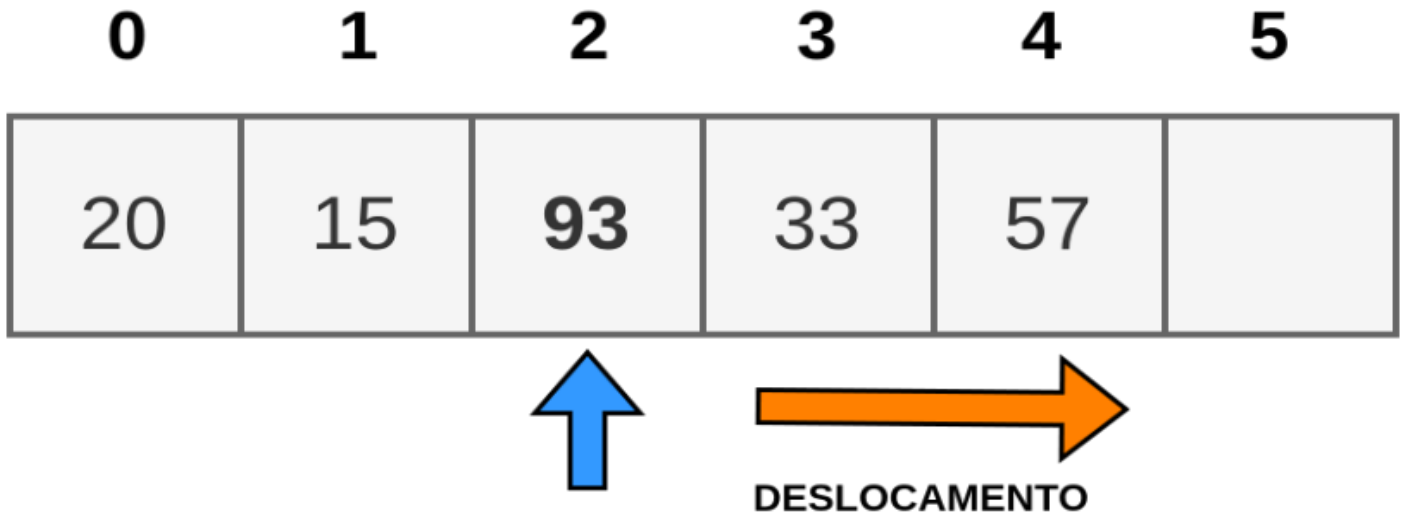
Listas Estáticas

- Inserção do **número 93** na **posição 2**.



Listas Estáticas

- Inserção do **número 93** na **posição 2**.



Listas Estáticas

- Durante o curso, utilizaremos ***struct*** para auxiliar na implementação das estruturas de dados que estudaremos.
- Dessa forma, podemos definir uma lista estática da seguinte forma:

```
#define TAMANHO_MAX 10

typedef struct tipo_lista_estatica {
    int itens[TAMANHO_MAX];
    int tamanho;
} ListaEstatica;
```

Listas Estáticas

- Durante o curso, utilizaremos ***struct*** para auxiliar na implementação das estruturas de dados que estudaremos.
- Dessa forma, podemos definir uma lista estática da seguinte forma:

```
#define MAX_SIZE 10

typedef struct static_list_type {
    int items[MAX_SIZE];
    int size;
} StaticList;
```

Listas Estáticas

- Podemos definir também as seguintes funções:

```
void inicializar(ListaEstatica *lista);
int esta_vazia(ListaEstatica lista);
int esta_cheia(ListaEstatica lista);
int tamanho(ListaEstatica lista);
int acessar(ListaEstatica lista, int indice, int *item);
int atualizar(ListaEstatica *lista, int indice, int item);
int inserir_item(ListaEstatica *lista, int item);
int inserir_em(ListaEstatica *lista, int indice, int item);
int remover_item(ListaEstatica *lista, int *item);
int remover_em(ListaEstatica *lista, int indice, int *item);
void mostrar(ListaEstatica lista);
```

Listas Estáticas

- Podemos definir também as seguintes funções:

```
void initialize(StaticList *list);
int is_empty(StaticList list);
int is_full(StaticList list);
int size(StaticList list);
int access(StaticList list, int index, int *item);
int update(StaticList *list, int index, int item);
int insert_item(StaticList *list, int item);
int insert_at(StaticList *list, int index, int item);
int remove_item(StaticList *list, int *item);
int remove_at(StaticList *list, int index, int *item);
void show(StaticList list);
```

Listas Estáticas

- Podemos definir também as seguintes funções:

```
int insert_at(StaticList *list, int index, int item) {
    if (index < 0 || index > list->size || is_full(*list)) {
        return 0;
    }

    for (int i = list->size; i > index; i--) {
        list->items[i] = list->items[i - 1];
    }

    list->items[index] = item;
    list->size++;

    return 1;
}
```

Alocação Dinâmica de um Array

- Embora os *arrays* sejam considerados estáticos devido à alocação contígua de endereços de memória, também é possível realizar a alocação em tempo de execução.
- Dessa forma, o usuário pode determinar o espaço que precisa para armazenar suas informações, e deixamos de utilizar uma reserva prévia arbitrária.
- Porém, independente da alocação ser feita de forma dinâmica, a lista não deixa de ser estática, visto que não ocorre de fato a inserção e remoção de nós.
- Em C, a alocação dinâmica utiliza a ***função malloc***.

Alocação Dinâmica de um Array

- Para alocar uma lista estática em tempo de execução, utilizamos:

```
printf("Insira o tamanho do vetor: ");
scanf("%d", &size);

if (size <= 0) {
    printf("Tamanho inválido\n");
    return 0;
}

int *vector = (int *) malloc(size * sizeof(int));
```

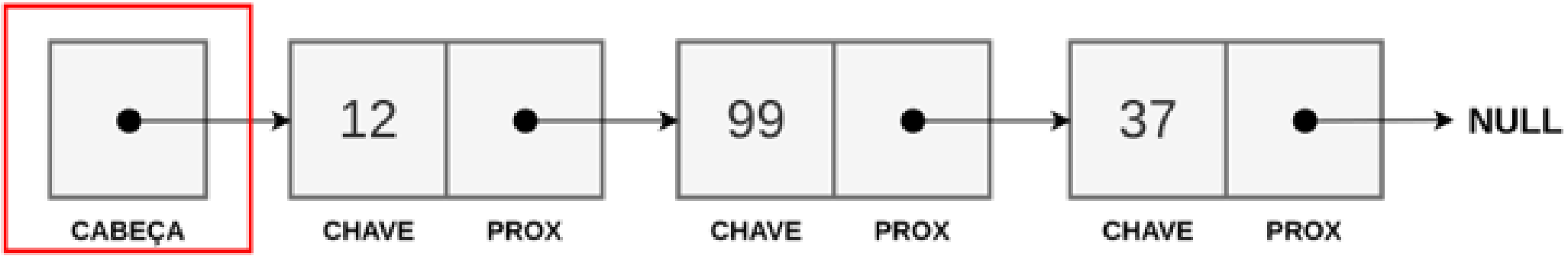
Alocação Encadeada

Listas Encadeadas

- Listas encadeadas (ou listas ligadas/linked lists) são compostas por uma **sequência de nós**, onde cada nó contém um **dado** e **pelo menos um ponteiro** para o próximo nó da lista.
- Dispõem os elementos em uma ordem linear, porém **não dependem de alocação prévia**.
- Nesse tipo de estrutura, os nós são alocados conforme necessário, **podendo estar em qualquer posição de memória**, por isso sua alocação pode ser denominada **dinâmica**.

Listas Encadeadas

- Dessa forma, em vez de armazenar apenas o dado, cada nó **também deve armazenar um endereço de memória (ponteiro) para o próximo nó da lista**.



Listas Encadeadas

- O acesso deixa de ser direto (por meio de índices) e passa a ser de forma **sequencial**, isto é, **é necessário percorrer nó a nó**.
- Em contrapartida, a inserção e a remoção de nós no início ou em posições arbitrárias da lista passam a ser mais eficientes.
    - Uma vez que não haverá deslocamento real, **somente manipulação de ponteiros**.

Listas Encadeadas

- Nessa implementação, para inserir ou remover, basta reajustar as ligações dos nós, alocando ou liberando o espaço de memória.
- Em relação ao espaço, há a desvantagem de armazenar o endereço do próximo nó junto a cada nó, mas, em compensação, somente os nós necessários são alocados.
- Operações como quebrar uma lista em sublistas ou juntar duas listas em uma única também são facilitadas, pois só dependem de ajustes nos ponteiros.

Listas Encadeadas

- Abaixo, definimos em C um tipo de nó (**No**) com um valor inteiro (**item**) e um endereço para o próximo nó (**prox**) que serve de base para implementar uma lista encadeada:

```
typedef struct no {
    int item;
    struct no *prox;
} *No;

void inicializar(No *cabeca) {
    *cabeca = NULL;
}
```

Listas Encadeadas

- Abaixo, definimos em C um tipo de nó (**Node**) com um valor inteiro (**item**) e um endereço para o próximo nó (**next**) que serve de base para implementar uma lista encadeada:

```
typedef struct node {
    int item;
    struct node *next;
} *Node;

void initialize(Node *head) {
    *head = NULL;
}
```

Listas Encadeadas

- Considerando o tipo de nó, podemos implementar as seguintes rotinas que nos permitem manipular uma lista encadeada:

```
int esta_vazia(No cabeca);
int inserir_inicio(No *cabeca, int item);
int inserir_fim(No *cabeca, int item);
int remover_inicio(No *cabeca, int *item);
int remover_fim(No *cabeca, int *item);
int contar_itens(No cabeca);
void mostrar(No cabeca);
```

Listas Encadeadas

- Considerando o tipo de nó, podemos implementar as seguintes rotinas que nos permitem manipular uma lista encadeada:

```
int is_empty(Node head);
int insert_front(Node *head, int item);
int insert_back(Node *head, int item);
int remove_front(Node *head, int *item);
int remove_back(Node *head, int *item);
int count_items(Node head);
void show(Node head);
```

Listas Encadeadas

- Com o tipo implementado, verificar se a lista está vazia requer simplesmente uma verificação de nulidade da cabeça da lista (**head**):

```
int is_empty(Node head) {
    return head == NULL;
}
```

Listas Encadeadas

- Inserir um item no início da lista também é uma operação simples, pois já temos acesso ao primeiro elemento da lista:

```
int insert_front(Node *head, int item) {
    Node new_node = (Node) malloc(sizeof(struct node));

    if (new_node == NULL) {
        return 0;
    }

    new_node->item = item;
    new_node->next = *head;

    *head = new_node;

    return 1;
}
```

Listas Encadeadas

- Para simplificar as inserções na lista, é possível definir uma rotina conveniente responsável pela criação de novos nós:

```
Node create_node(int item) {
    Node new_node = (Node) malloc(sizeof(struct node));

    if (new_node != NULL) {
        new_node->item = item;
        new_node->next = NULL;
    }

    return new_node;
}
```

Listas Encadeadas

- Utilizando a rotina de criação de nós, temos a seguinte rotina de inserção no início:

```
int insert_front(Node *head, int item) {
    Node new_node = create_node(item);

    if (new_node == NULL) {
        return 0;
    }

    new_node->next = *head;

    *head = new_node;

    return 1;
}
```

Listas Encadeadas

- Uma desvantagem dessa definição de lista encadeada é que a inserção ou remoção no fim da lista requer que a lista seja percorrida até o último item;

```
int insert_back(Node *head, int item) {
    Node new_node = (Node) malloc(sizeof(struct node));

    if (new_node == NULL) {
        return 0;
    }

    new_node->item = item;
    new_node->next = NULL;

    if (*head == NULL) {
        *head = new_node;
        return 1;
    }

    Node temp = *head;

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = new_node;

    return 1;
}
```

Listas Encadeadas

- Como não temos um indicador do tamanho nessa implementação, precisamos percorrer os items até encontrar um nulo sinalizando o fim;

```
int count_items(Node head) {
    if (is_empty(head)) {
        return 0;
    }

    int count = 0;

    while (head != NULL) {
        count++;
        head = head->next;
    }

    return count;
}
```

Variações

- A lista apresentada até então é a versão mais simples de lista encadeada, que é denominada **lista simplesmente encadeada**.
- Porém, existem variações mais robustas que podemos implementar:
  - **Lista Simplesmente Encadeada com Cabeça e Cauda**
  - **Lista Duplamente Encadeada**
  - **Lista Circular Duplamente Encadeada**

Com Cabeça e Cauda

- Ao implementar uma lista encadeada, podemos criar um segundo *struct* para armazenar as informações gerais da lista, como cabeça, cauda e tamanho:

```
typedef struct no {
    int item;
    struct no *prox;
} *No;

typedef struct lista {
    No cabeca, cauda;
    int tamanho;
} Lista;
```

Com Cabeça e Cauda

- Ao implementar uma lista encadeada, podemos criar um segundo *struct* para armazenar as informações gerais da lista, como cabeça (**head**), cauda (**tail**) e tamanho (**size**):

```
typedef struct node {
    int item;
    struct node *next;
} *Node;

typedef struct list {
    Node head, tail;
    int size;
} List;
```

Com Cabeça e Cauda

- Dessa forma, não precisamos mais realizar a contagem para determinar o número de elementos na lista e a implementação da função que verifica vazio pode ser feita assim:

```
int is_empty(List list) {
    return list.size == 0;
}
```

Com Cabeça e Cauda

- Como agora temos um ponteiro para a cauda, inserções ao fim da lista podem ser feitas sem percorrer toda a lista;
  - Note que o tamanho deve ser atualizado a cada inserção ou remoção.

```
int insert_back(List *list, int item) {
    Node new_node = (Node) malloc(sizeof(struct node));

    if (new_node == NULL) {
        return 0;
    }

    new_node->item = item;
    new_node->next = NULL;

    if (list->size == 0) {
        list->head = list->tail = new_node;
        list->size++;
        return 1;
    }

    list->tail->next = new_node;
    list->tail = new_node;

    list->size++;

    return 1;
}
```

Com Cabeça e Cauda

- Porém, apesar do ponteiro para a cauda, remover o último nó da lista ainda requer o percorrimento dos nós, já que é preciso acessar o penúltimo nó;

```
int remove_back(List *list, int *item) {
    if (is_empty(*list)) {
        return 0;
    }

    *item = list->tail->item;

    if (list->size == 1) {
        free(list->tail);
        list->head = list->tail = NULL;
        list->size--;
        return 1;
    }

    Node temp = list->head;

    while (temp->next != list->tail) {
        temp = temp->next;
    }

    free(list->tail);
    temp->next = NULL;
    list->tail = temp;
    list->size--;

    return 1;
}
```

Lista Duplamente Encadeada

- Para termos mais flexibilidade, além de manter um ponteiro para o próximo elemento, podemos também manter um segundo ponteiro com o endereço do nó anterior (**ant**);

```
typedef struct no {
    int item;
    struct no *ant, *prox;
} *No;

typedef struct lista {
    No cabeca, cauda;
    int tamanho;
} Lista;


void inicializar(Lista *lista) {
    lista->cabeca = lista->cauda = NULL;
    lista->tamanho = 0;
}
```

Lista Duplamente Encadeada

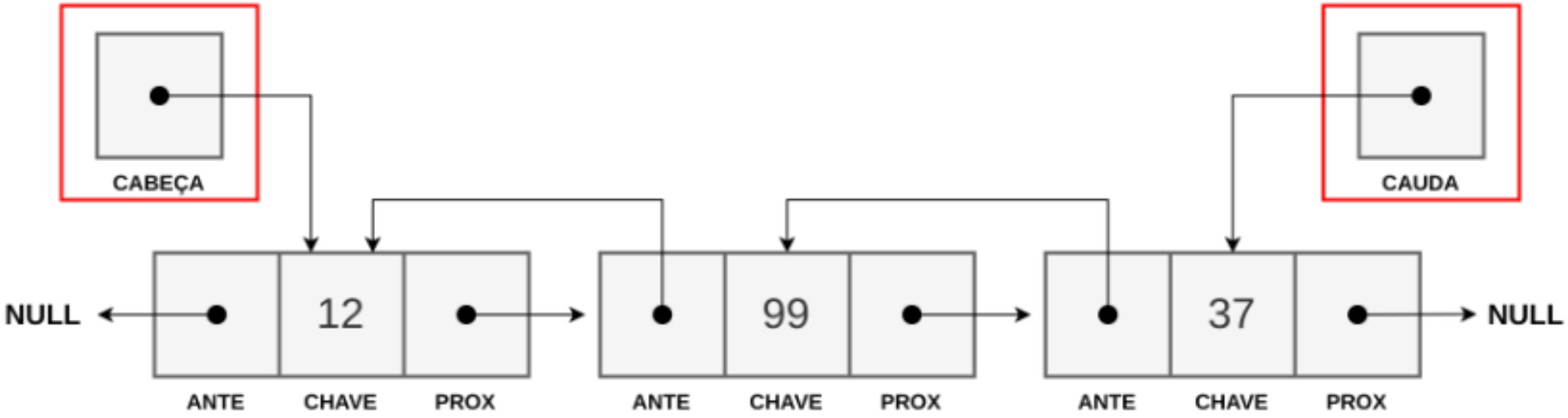
- Para termos mais flexibilidade, além de manter um ponteiro para o próximo elemento, podemos também manter um segundo ponteiro com o endereço do nó anterior (**prev**);

```
typedef struct node {
    int item;
    struct node *prev, *next;
} *Node;

typedef struct list {
    Node head, tail;
    int size;
} List;


void initialize(List *list) {
    list->head = list->tail = NULL;
    list->size = 0;
}
```

Lista Duplamente Encadeada



Lista Duplamente Encadeada

- Com essa estrutura, podemos definir as mesmas funções que anteriormente, mas também rotinas extras como uma para mostrar a lista em ordem inversa:

```
int esta_vazia(Lista lista);
int inserir_inicio(Lista *lista, int item);
int inserir_fim(Lista *lista, int item);
int remover_inicio(Lista *lista, int *item);
int remover_fim(Lista *lista, int *item);
int espiar_inicio(Lista lista, int *item);
int espiar_fim(Lista lista, int *item);
void mostrar(Lista lista);
void mostrar_reverso(Lista lista);
```

Lista Duplamente Encadeada

- Com essa estrutura, podemos definir as mesmas funções que anteriormente, mas também rotinas extras como uma para mostrar a lista em ordem inversa;
- Note que podemos chamar funções de inserção como **push** e de remoção como **pop:**

```
int is_empty(List list);
int push_front(List *list, int item);
int push_back(List *list, int item);
int pop_front(List *list, int *item);
int pop_back(List *list, int *item);
int peek_front(List list, int *item);
int peek_back(List list, int *item);
void show(List list);
void show_reverse(List list);
```

Lista Duplamente Encadeada

- Com o ponteiro para o nó anterior, a remoção do último nó passa a ser facilitada, pois podemos acessar o penúltimo nó através da cauda da lista;

```
int pop_back(List *list, int *item) {
    if (list->size == 0) {
        return 0;
    }

    *item = list->tail->item;

    Node p = list->tail;
    list->tail = p->prev;
    free(p);

    if (list->tail == NULL) {
        list->head = NULL;
    } else{
        list->tail->next = NULL;
    }

    list->size--;

    return 1;
}
```

Lista Duplamente Encadeada

- Para mostrar a lista em ordem inversa, basta acessar pela cauda e percorrer utilizando os ponteiros para os nós anteriores até atingir a cabeça da lista, com anterior nulo.

```
void show_reverse(List list) {
    if (list.size == 0) {
        return;
    }

    Node p = list.tail;

    printf("List items in reverse order: ");

    while (p != NULL) {
        printf("%d ", p->item);
        p = p->prev;
    }

    printf("\n");
}
```

Lista Duplamente Encadeada

- Embora não tenhamos feito anteriormente, podemos definir rotinas convenientes para o acesso dos valores contidos no primeiro (**head**) e último (**tail**) nós;

```
int peek_front(List list, int *item) {
    if (is_empty(list)) {
        return 0;
    }

    *item = list.head->item;

    return 1;
}

int peek_back(List list, int *item) {
    if (is_empty(list)) {
        return 0;
    }

    *item = list.tail->item;

    return 1;
}
```

Lista Circular Duplamente Encadeada

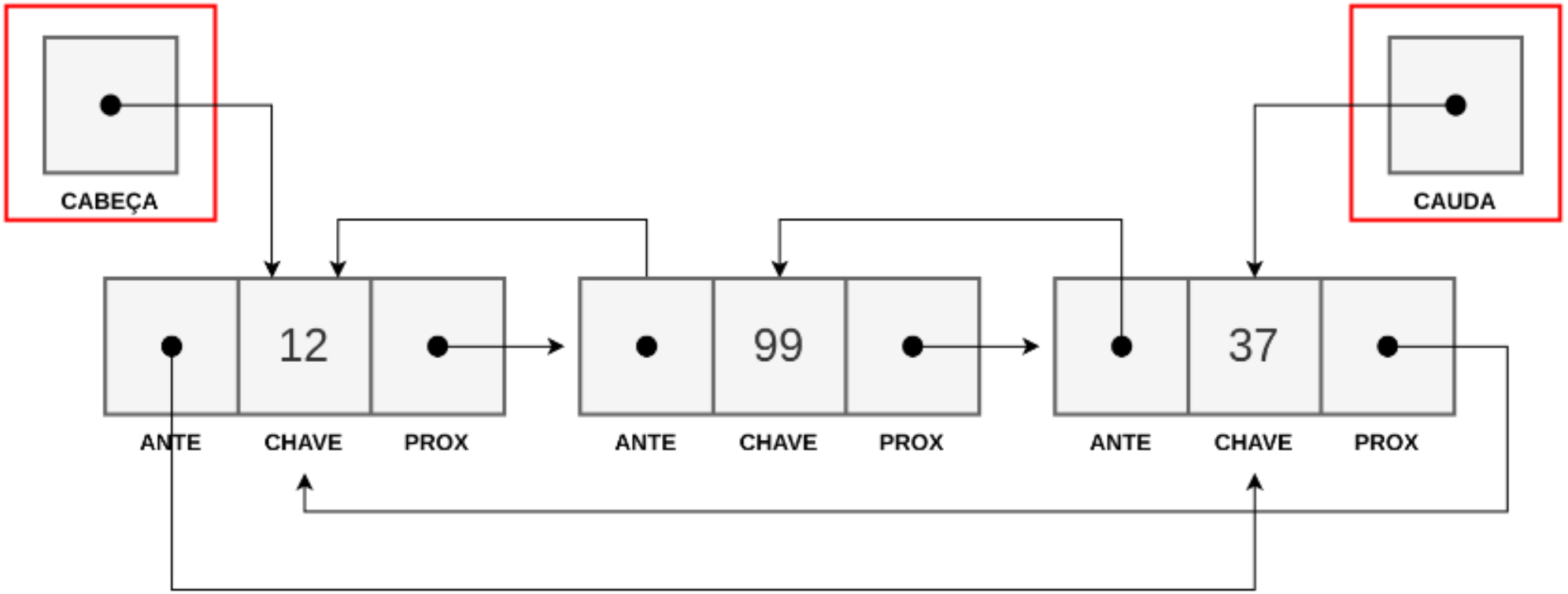
- Um último tipo de lista que podemos implementar é a circular;
- A vantagem dessa lista quando duplamente encadeada é que ela dispensa a necessidade de um ponteiro para o último elemento da lista;
- A cabeça da lista dá acesso ao último nó;

```
typedef struct node {
    int item;
    struct node *prev, *next;
} *Node;

typedef struct list {
    Node head;
    int size;
} List;

void initialize(List *list) {
    list->head = NULL;
    list->size = 0;
}
```

Lista Circular Duplamente Encadeada



Lista Circular Duplamente Encadeada

- A inserção na frente da lista agora requer o cuidado de atualizar as referências que apontam ao último nó da lista e também as referências desse último ao início;

```
int push_front(List *list, int item) {
    Node new_node = (Node) malloc(sizeof(struct node));

    if (new_node == NULL) {
        return 0;
    }

    new_node->item = item;

    if (is_empty(*list)) {
        new_node->next = new_node->prev = new_node;
    } else {
        new_node->next = list->head;
        new_node->prev = list->head->prev;

        list->head->prev->next = new_node;
        list->head->prev = new_node;
    }

    list->head = new_node;

    list->size++;

    return 1;
}
```

Lista Circular Duplamente Encadeada

- De forma análoga, a inserção ao final da lista também requer o cuidado de não só alterar os endereços do antepenúltimo nó, mas também do primeiro nó da lista;

```
int push_back(List *list, int item) {
    Node new_node = (Node) malloc(sizeof(struct node));

    if (new_node == NULL) {
        return 0;
    }

    new_node->item = item;

    if (is_empty(*list)) {
        new_node->next = new_node->prev = new_node;
        list->head = new_node;
    } else {
        new_node->prev = list->head->prev;

        list->head->prev->next = new_node;
        list->head->prev = new_node;

        new_node->next = list->head;
    }

    list->size++;

    return 1;
}
```

Complexidade

Complexidade

Operação	Lista estática (vetor)	Simples	Simples (início + fim)	Dupla (início + fim)	Circular dupla
Inserir no começo	O(n)	O(1)	O(1)	O(1)	O(1)
Inserir no fim	O(1)	O(n)	O(1)	O(1)	O(1)
Remover do começo	O(n)	O(1)	O(1)	O(1)	O(1)
Remover do fim	O(1)	O(n)	O(n)	O(1)	O(1)
Inserir em posição arbitrária	O(n)	O(n)	O(n)	O(n)	O(n)
Remover em posição arbitrária	O(n)	O(n)	O(n)	O(n)	O(n)
Percorrer a lista	O(n)	O(n)	O(n)	O(n)	O(n)
Acessar elemento qualquer	O(1)	O(n)	O(n)	O(n)	O(n)

Ficou alguma dúvida?

E-mail para contato: giovani.candido@unesp.br e lf.sementille@unesp.br

Materiais e Avisos: Google Classroom